

iAM-Scout: Ein datengetriebenes Scouting-Tool für Schweizer Amateurfussball

Modul: Big Data Projects

Fabian Meier

meierf07@students.zhaw.ch
ZHAW School of Engineering

Cedric Niklaus

niklaced@students.zhaw.ch
ZHAW School of Engineering



Bildquelle: Unsplash [1]

Abstract

Data-driven scouting is widely used in professional football, but it is still hardly accessible in Swiss amateur football. iAM-Scout addresses this gap with a prototype that collects, processes, and makes publicly available football data usable for scouting purposes. Amateur data from Transfermarkt as well as professional performance data and ratings from Sofascore are cleaned, standardized, and stored in a structured data pipeline. Based on the professional data, a machine learning model is trained to estimate player ratings and transfer them to amateur players. The system includes scraping, data preprocessing, feature engineering, a database, an API, and a frontend. The resulting database contains 4'891 matches and 146'964 player performance records. The best rating model achieved a MAE of 0.226 and a RMSE of 0.333. As a result, the prototype provides a basis for comparing amateur players with data-driven methods and identifying potential transfer candidates. At the same time, the project shows limitations regarding data quality, dependency on external websites, and the maintenance effort required for web scrapers.

1 Einleitung

Im Fussball spielt Scouting eine zentrale Rolle, um passende Spieler zu identifizieren, Transfers vorzubereiten und langfristig die sportliche Qualität eines Vereins zu verbessern. Während professionelle Vereine häufig auf spezialisierte Analyseplattformen, detaillierte Statistiken und eigene Scouting-Abteilungen zurückgreifen können, stehen Amateurvereinen deutlich weniger Ressourcen zur Verfügung. Gleichzeitig steckt gerade im Amateurfussball viel ungenutztes Potenzial: Spieler können sich weiterentwickeln, höhere Ligen erreichen oder durch gezielte Förderung langfristig auch für den Profifussball relevant werden.

Im Schweizer Amateurfussball ist datengetriebenes Scou-

ting jedoch nur eingeschränkt möglich. Zwar werden zentrale Qualifikations- und Transferprozesse im Schweizer Fussball seit 2019 über clubcorner.ch abgewickelt [2], diese Daten stehen jedoch nicht als offene, direkt nutzbare Scouting-Datenbank zur Verfügung. Für iAM-Scout wurde deshalb der Weg über öffentlich zugängliche Daten von Transfermarkt gewählt, da dort Informationen zu Vereinen, Spielern, Spielen und Leistungsdaten einfacher verfügbar sind [3]. Dennoch fehlen im Amateurbereich meist vergleichbare Spielerbewertungen, wie sie im Profifussball beispielsweise durch Bewertungssysteme bekannt sind. Auch detaillierte Leistungskennzahlen wie erwartete Tore, Passqualität oder umfangreiche Eventdaten sind in diesem Bereich in der Regel nicht öffentlich vorhanden. Dadurch basieren Transfer- und Kaderentscheidungen in Amateurvereinen oft stärker auf Beobachtungen, persönlichen Netzwerken oder subjektiven Einschätzungen.

Das Projekt iAM-Scout adressiert diese Lücke durch einen Prototyp für datengetriebenes Scouting im Schweizer Amateurfussball. Als Grundlage werden Amateurdaten von Transfermarkt sowie professionelle Leistungsdaten und Bewertungen von Sofascore verwendet [3], [4]. Die Grundidee besteht darin, ein Machine-Learning-Modell auf Profidaten zu trainieren, bei denen Spielerbewertungen vorhanden sind, und dieses Modell anschliessend auf Amateurspieler anzuwenden, für die solche Bewertungen normalerweise nicht existieren. Dadurch sollen Spieler vergleichbarer gemacht und datenbasierte Hinweise für potenzielle Transfers erzeugt werden.

Aus technischer Sicht besteht iAM-Scout aus mehreren Komponenten. Zunächst werden Daten durch Scraping gesammelt und anschliessend bereinigt, vereinheitlicht und in einer relationalen Datenbank gespeichert. Darauf aufbauend wird ein Modell zur Schätzung von Spielerbewertungen trainiert. Die Ergebnisse werden über ein Backend bereitgestellt und in einem Frontend nutzbar gemacht, sodass Informationen zu Teams, Spielern, Ligen und Spielen abgefragt werden können. Zusätzlich wurde ein Recommender-System umgesetzt,

das Vereine bei der Suche nach passenden Transferkandidaten unterstützen soll.

Der Beitrag dieser Arbeit liegt damit nicht nur in einem einzelnen Modell, sondern in einer durchgängigen Datenpipeline vom Datenbezug bis zur nutzbaren Anwendung. Das Projekt verbindet Web Scraping, Datenaufbereitung, Datenbankdesign, Machine Learning, API-Entwicklung und Frontend-Visualisierung zu einem Prototyp, der zeigt, wie datengetriebenes Scouting auch für den Schweizer Amateurfussball zugänglich gemacht werden kann.

2 Hintergrund und Problemstellung

Aufbauend auf der Motivation aus der Einleitung liegt der Fokus dieses Kapitels auf der technischen Problemstellung. Für iAM-Scout reicht es nicht, einzelne Spielerdaten manuell anzuschauen. Die Daten müssen automatisiert gesammelt, strukturiert gespeichert, aktualisiert und später für Modellierung und Empfehlungen verwendet werden können.

Als fachlicher Rahmen wurden aktuell die Promotion League sowie die drei Gruppen der 1. Liga betrachtet. Damit entsteht ein Datenausschnitt des Schweizer Amateurfussballs, der über mehrere Saisons hinweg analysiert werden kann. Die Datenbasis besteht nicht nur aus einzelnen Spielern, sondern aus mehreren zusammenhängenden Entitäten. Dazu gehören Mannschaften, Liga-Zugehörigkeiten von Clubs, Spielerprofile, Kader pro Saison, Spiele sowie Spielerleistungen in einzelnen Spielen. Diese Struktur ist wichtig, weil Spieler über Saisons hinweg den Verein wechseln können und Leistungen immer im Kontext von Spiel, Team, Liga und Saison betrachtet werden müssen.

Ein zentrales Problem ist, dass die benötigten Informationen zwar öffentlich sichtbar sind, aber nicht als fertige Datenbank oder API bereitstehen. Die Daten mussten deshalb aus Webseiten extrahiert und in eine eigene relationale Struktur übertragen werden. Transfermarkt war dafür grundsätzlich gut geeignet, weil viele Seiten über IDs in den URLs systematisch erreichbar sind [3]. Trotzdem lagen die Inhalte zuerst als HTML vor und mussten mit Scraping-Logik ausgelesen werden. Dadurch wurde das Scraping zum aufwendigsten Teil des Projekts. Es war zeitintensiv, teilweise schwer vorhersehbar und erforderte zusätzliche Fehlerbehandlung.

Damit die Datenerfassung über längere Läufe stabil bleibt, mussten Delays, Retries und Logging eingebaut werden. Der Scraper protokolliert dabei, wann welche Schritte ausgeführt wurden und wie viele neue Einträge hinzugekommen sind. Das ist wichtig, weil bei Web Scraping einzelne Seiten fehlschlagen können oder sich die Struktur einer externen Webseite ändert. Gerade gegen Ende des Projekts zeigte sich diese Abhängigkeit deutlich, da Anpassungen an der Webseite direkte Auswirkungen auf den Scraper hatten.

Für die Modellierung kommt zusätzlich eine zweite Datenquelle dazu. Da für Amateurspieler keine direkten Spielerbewertungen vorhanden sind, werden Profidaten verwendet, bei denen Bewertungen verfügbar sind. Diese Bewertungen stammen von Sofascore [4]. Die Profildaten und Leistungsdaten werden mit den Bewertungen zusammengeführt, damit

ein Modell auf Profispielern trainiert werden kann. Anschliessend wird dasselbe Modell auf Amateurspieler angewendet, weil dort vergleichbare Feature-Spalten vorhanden sind. Die Schwierigkeit liegt dabei vor allem im sauberen Zusammenführen der Quellen, zum Beispiel bei Spielernamen oder uneinheitlichen Profilinformatoren.

Aus Big-Data-Sicht geht es bei iAM-Scout weniger um Datenmengen im Bereich von vielen Gigabytes, sondern um eine grössere Anzahl strukturierter Einträge, mehrere Tabellen, wiederkehrende Updates und unterschiedliche Datenquellen. Besonders relevant ist die Aktualität: Da laufend neue Spiele stattfinden, muss die Pipeline nicht nur historische Daten laden, sondern auch neue Spiele inkrementell ergänzen können. Dabei dürfen keine Duplikate entstehen und bestehende Beziehungen zwischen Spielern, Teams, Spielen und Saisons müssen erhalten bleiben.

Die Problemstellung lässt sich deshalb als Pipeline-Problem beschreiben. Aus öffentlich sichtbaren, aber nicht direkt nutzbaren Webdaten soll eine konsistente Datenbasis entstehen. Diese Datenbasis bildet die Grundlage für Abfragen, Modellvorhersagen und ein Recommender-System. Am Ende soll das System nicht nur einzelne Informationen anzeigen, sondern den ganzen Weg vom Scraping bis zur Anwendung abdecken.

3 Systemdesign

Die Systemarchitektur von iAM-Scout ist als durchgängige Datenpipeline aufgebaut. Das System sammelt Daten aus externen Webseiten, verarbeitet diese in einer ETL-Pipeline, speichert sie in einer PostgreSQL-Datenbank und stellt sie anschliessend über eine API und ein Frontend bereit. Abbildung 1 zeigt die wichtigsten Komponenten und deren Zusammenspiel.

Für das Scraping werden je nach Webseite und Anwendungsfall `requests`, `BeautifulSoup` und `Playwright` verwendet. Einfachere Seiten können direkt über HTTP-Requests geladen und mit `BeautifulSoup` geparkt werden. Für dynamischere Inhalte wird `Playwright` eingesetzt, da damit Webseiten wie in einem Browser gerendert werden können. Die Scraper liefern die Rohdaten für die weitere Verarbeitung.

Die Orchestrierung der Pipeline erfolgt mit `Apache Airflow`. Dabei gibt es zwei zentrale DAGs. Der erste DAG läuft theoretisch jeweils am Montagmorgen und lädt neue Spiele vom Wochenende nach. Dabei werden nur neue oder fehlende Einträge ergänzt. Der zweite DAG läuft jeweils am 1. August und aktualisiert die Club-Zugehörigkeiten für die neue Saison. Spieler, Kader und Spielerleistungen werden im wöchentlichen Prozess ergänzt, da sich diese Daten auch während der Saison ändern können.

Nach dem Scraping durchlaufen die Daten eine ETL-Pipeline. In diesem Schritt werden die Daten an das Datenbankschema angepasst, validiert und für die Speicherung vorbereitet. Zusätzlich werden die beiden trainierten Modellen auf neue Amateurdaten angewendet. Die Modelle wurden trainiert und anschliessend als `Pickle-Datei` gespeichert. Sie werden im laufenden Prozess nur geladen und angewendet, aber nicht automatisch neu trainiert.

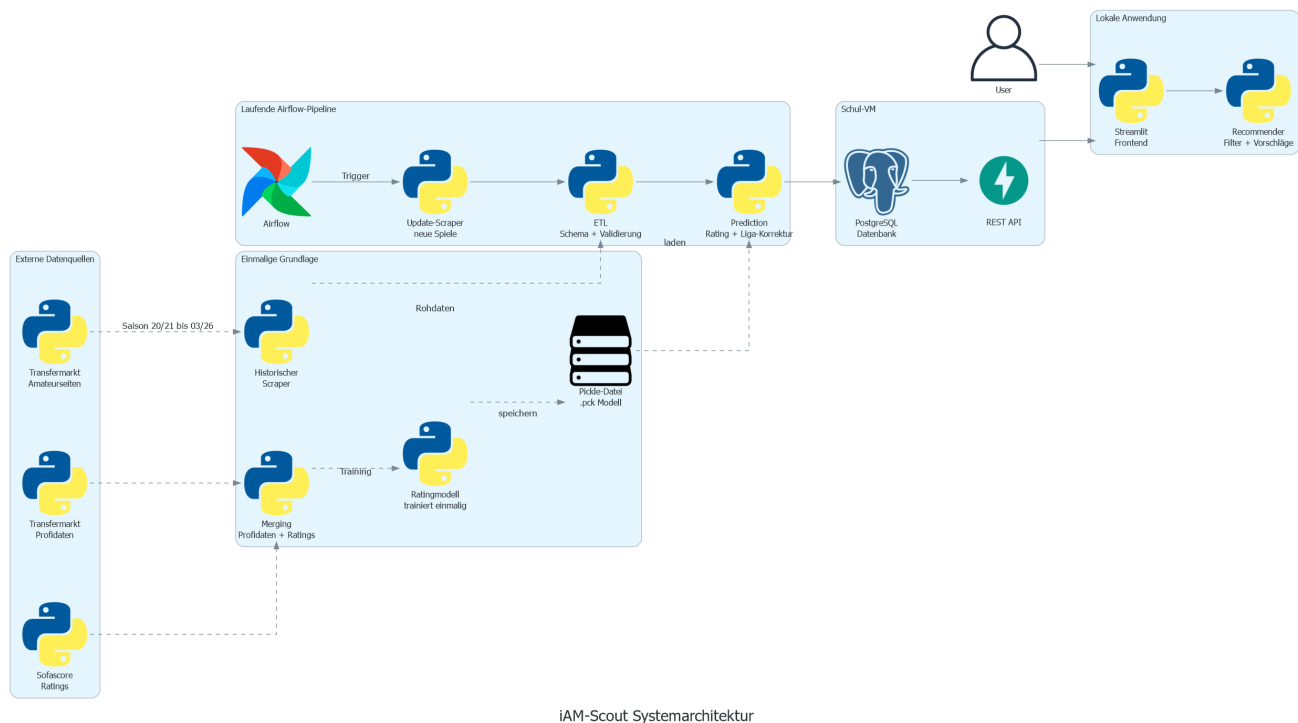


Abbildung 1: Systemarchitektur von iAM-Scout

Die PostgreSQL-Datenbank läuft zusammen mit Airflow und der REST-API auf einer von der Schule bereitgestellten virtuellen Maschine. Die Pipeline schreibt direkt in die Datenbank. Die API ist nur für lesende Zugriffe aus dem Frontend zuständig und kapselt damit den direkten Datenbankzugriff. Das Streamlit-Frontend wird lokal ausgeführt und kommuniziert über die FastAPI-Schnittstelle mit der Datenbank. Dadurch bleibt die Anwendung klar getrennt: Die Pipeline ist für Datenaufnahme und Verarbeitung verantwortlich, die API für den Zugriff und das Frontend für die Darstellung.

Das Datenbankschema ist in Abbildung 2 dargestellt. Die Tabellen sind `clubs`, `players`, `clubs_per_season`, `matches`, `squads` und `player_stats`. Die Tabelle `clubs_per_season` ist dabei zentral, weil ein Club je nach Saison in einer anderen Liga spielen kann. Über `squads` wird abgebildet, welche Spieler in welcher Saison zu welchem Club gehören. Die Tabelle `matches` speichert die Spiele, während `player_stats` die einzelnen Spielerleistungen pro Spiel enthält.

Die Vermeidung von Duplikaten wird hauptsächlich über das Datenbankschema gelöst. Primärschlüssel und zusammengesetzte Schlüssel stellen sicher, dass zum Beispiel ein Spieler, ein Spiel, eine Kaderzugehörigkeit oder eine Spielerleistung nicht mehrfach identisch gespeichert werden kann. Zusätzlich unterstützen Indizes häufige Abfragen, zum Beispiel nach Spielen eines Clubs, Saisons oder Spielerleistungen.

Ein wichtiger Designentscheid war die Trennung zwischen historischem Import und laufendem Update. Zu Beginn wurden die Amateurdaten von der Saison 2020/21 bis März 2026 einmalig geladen. Danach soll das System inkrementell weitergeführt werden. Dadurch muss nicht bei jedem Lauf der

komplette Datenbestand neu verarbeitet werden. Das spart Zeit und reduziert unnötige Zugriffe auf externe Webseiten.

Zur Nachvollziehbarkeit werden sowohl Airflow-Logs als auch eigene Scraper-Logs verwendet. Airflow zeigt, welche Pipeline-Schritte erfolgreich waren oder fehlgeschlagen sind. Die Scraper protokollieren zusätzlich, welche Schritte sie ausführen und wie viele neue Einträge gefunden wurden. Das ist wichtig, weil Web Scraping fehleranfällig ist und externe Webseiten ihre Struktur ändern können.

Insgesamt ist das System so aufgebaut, dass die einzelnen Komponenten klar getrennt bleiben. Scraping, Verarbeitung, Modellanwendung, Speicherung, API und Frontend können dadurch unabhängig voneinander angepasst werden. Der wichtigste Trade-off ist die Abhängigkeit von externen Webseiten. Dafür musste kein eigener Datensatz manuell gepflegt werden, aber Änderungen an Transfermarkt oder Sofascore können Anpassungen an den Scrapern notwendig machen.

4 Experimentelle Evaluation

Die Evaluation von iAM-Scout besteht aus drei Teilen. Zuerst wird geprüft, ob die Datenpipeline eine ausreichende und konsistente Datenbasis aufbauen konnte. Danach werden die beiden Modelle quantitativ bewertet. Abschließend wird das Recommender-System qualitativ mit Fußball-Domänenwissen geprüft, da nicht alle Empfehlungen rein über Metriken beurteilt werden können.

Die Experimente wurden mit Python durchgeführt. Für die Modellierung wurden scikit-learn verwendet, während die Daten in PostgreSQL gespeichert und über die zuvor beschriebene

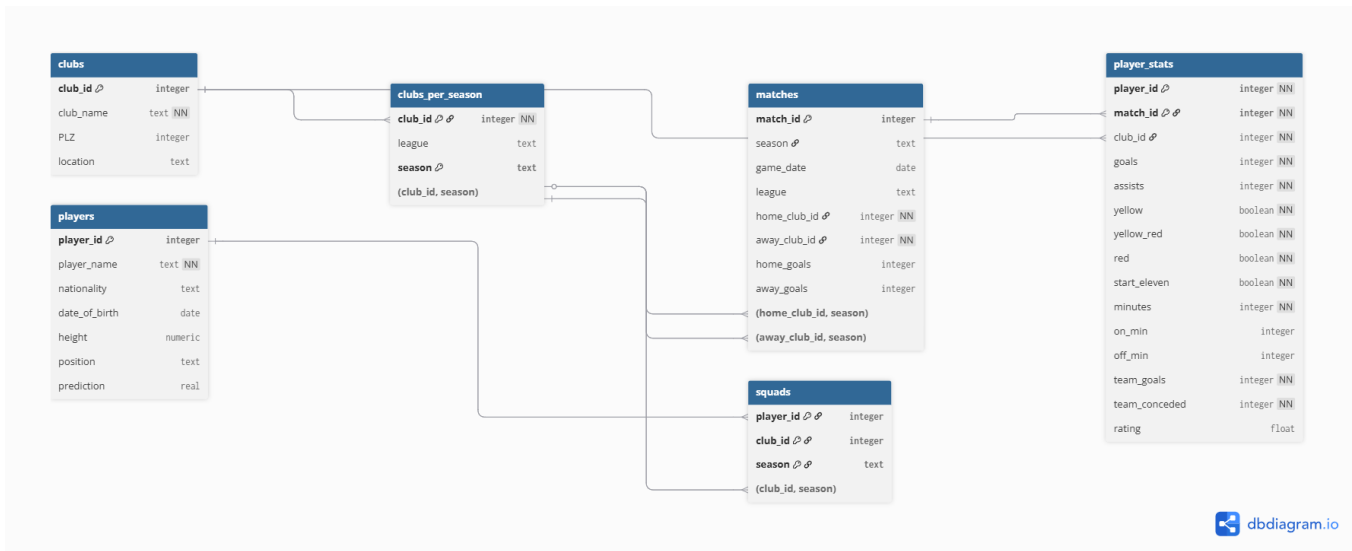


Abbildung 2: Relationales Datenbankschema von iAM-Scout

Tabelle 1: Scraping-Laufzeiten des historischen Imports

Liga	Spiele	Spieler	Spielerleist.	Laufzeit
PL	1'460	1'952	38'582	9h 01min
1. Liga Gr. 2	1'144	1'739	31'308	5h 13min
1. Liga Gr. 1	1'141	1'714	30'956	6h 03min
1. Liga Gr. 3	1'146	1'644	31'648	6h 23min

ne Pipeline vorbereitet wurden. Laufzeiten wurden vor allem über Scraper- und Airflow-Logs nachvollzogen.

4.1 Evaluation der Datenpipeline

Für den initialen Aufbau der Datenbank wurden die vier betrachteten Ligen historisch gescraped. Dabei zeigte sich, dass vor allem das Scraping der Spielerleistungen zeitintensiv ist. Dies liegt daran, dass für Spielerleistungen viele einzelne Spiel- und Spielerdetailseiten verarbeitet werden müssen. Tabelle 1 zeigt eine Zeitübersicht des historischen Imports.

Die Laufzeiten zeigen, dass das Problem weniger in der reinen Datenmenge in Gigabytes liegt, sondern in der Anzahl benötigter Webzugriffe und der Verarbeitung vieler einzelner HTML-Seiten. Die Scraper-Logs waren dabei wichtig, um nachvollziehen zu können, welche Schritte ausgeführt wurden und wie viele neue Einträge jeweils entstanden sind.

Der aktuelle Datenbankstand nach dem historischen Import und zusätzlichen Updates ist in Tabelle 2 dargestellt. Die meisten Einträge entstehen in der Tabelle `player_stats`, da dort für jedes Spiel mehrere Spielerleistungen gespeichert werden.

Zur Plausibilitätsprüfung wurde kontrolliert, ob die Anzahl Spiele und Leistungsdaten zu den erwarteten Ligastrukturen passen. Zusätzlich wurde geprüft, ob die Datenbank durch Primary Keys und zusammengesetzte Schlüssel Duplikate verhindert. Dadurch konnte der historische Import mit dem späteren inkrementellen Update-Prozess kombiniert werden, ohne bestehende Einträge mehrfach zu speichern.

Tabelle 2: Umfang der aufgebauten Datenbank

Tabelle	Anzahl Zeilen
clubs	93
clubs_per_season	380
matches	4'891
player_stats	146'964
players	5'602
squads	13'030

4.2 Evaluation des Ratingmodells

Das erste Modell schätzt eine Sofascore-ähnliche Bewertung für einzelne Spielerleistungen. Trainiert wurde es auf Profidaten, weil dort die Zielvariable, also die Spielerbewertung, verfügbar ist. Als Eingabevariablen wurden nur Merkmale verwendet, die auch für Amateurspieler vorliegen. Dazu gehören Tore, Assists, Minuten, Teamtore, Gegentore, Position, Spielresultat, gelbe und rote Karten sowie Startelfeinsatz.

Der Datensatz umfasste 12'340 Spielerleistungs-Zeilen. Für die Evaluation wurde ein 80/20-Split verwendet. Dabei wurde darauf geachtet, dass ein Spiel entweder vollständig im Trainings- oder vollständig im Testdatensatz liegt. Dadurch wird verhindert, dass Leistungen aus demselben Spiel gleichzeitig in Training und Test vorkommen.

Für den Random Forest wurde eine Pipeline mit vorgelagertem Preprocessing verwendet. Numerische und boolesche Variablen wurden direkt übernommen, während kategoriale Variablen wie Position und Spielresultat per One-Hot-Encoding verarbeitet wurden. Die Hyperparameter des Random Forest wurden mit einer Grid Search und 5-facher Cross-Validation optimiert. Als Optimierungskriterium wurde der RMSE verwendet. Die besten Hyperparameter sind im Anhang B aufgeführt.

Wie Tabelle 3 zeigt, erreichte der Random Forest die besten Resultate. Das Modell erzielte einen MAE von 0.182, einen RMSE von 0.306 und ein R^2 von 0.44. Ein durchschnittlicher

Tabelle 3: Vergleich der getesteten Ratingmodelle

Modell	MAE	RMSE
Lineare Regression	0.318	0.429
Random Forest	0.182	0.306
Ridge Regression	0.318	0.429
Lasso Regression	0.366	0.496
Neurales Netzwerk	0.300	0.411

Tabelle 4: Datengrundlage des Next-Season-Modells

Positionsgruppe	Zeilen	Train-Spieler	Test-Spieler
Torhüter	459	176	44
Verteidiger	1'981	691	173
Mittelfeld	1'879	684	171
Angreifer	1'324	503	126

Fehler von rund 0.18 Ratingpunkten ist für eine erste datenbasierte Einschätzung nutzbar, sollte aber nicht als exakte Bewertung einzelner Spieler verstanden werden. Gleichzeitig zeigt das R^2 , dass das Modell die Bewertungen nur teilweise erklären kann. Das ist plausibel, da viele Faktoren einer Spielerbewertung, zum Beispiel Zweikampfverhalten, Laufwege oder Passqualität, in den verfügbaren Amateurdaten nicht vorhanden sind. Anschliessend wurde das Random-Forest-Modell zusätzlich mit einem Match-Split trainiert. Dabei wurde sichergestellt, dass dieselben Spiele nicht gleichzeitig im Trainings- und Testdatensatz vorkommen, um eine realistischere Evaluation zu erhalten. Der RMSE stieg dabei auf 0.47, was zu erwarten war, da diese Aufteilung weniger Datenleakage zulässt und das Modell dadurch stärker auf unbekannten Spielen getestet wird.

4.3 Evaluation des Recommender-Modells

Für das Recommender-System wurde ein zweites Modell trainiert. Dieses Modell bewertet nicht einzelne Spielerleistungen, sondern sagt eine erwartete Bewertung für die nächste Saison voraus. Dafür wurden die Leistungsdaten pro Spieler und Saison aggregiert. Ein Datenpunkt beschreibt somit einen Spieler-Saison-Übergang, zum Beispiel von Saison 2023/24 auf 2024/25. Ein Spieler kann mehrere Datenpunkte erzeugen, wenn er über mehrere aufeinanderfolgende Saisons in den betrachteten Ligen gespielt hat.

Die Zielvariable ist `rating_next_season`. Als Features wurden unter anderem Alter, Position, durchschnittliches aktuelles Rating, Einsatzquote, Tore, Assists, Karten, Startelfanteil, Minuten, Anzahl Spiele, Teamtore, Gegentore sowie Sieg-, Unentschieden- und Niederlagenanteile verwendet. Zusätzlich wurden Informationen zur Ligastufe über Variablen wie `prozent_1_liga` und `prozent_pl` berücksichtigt.

Für das Modell wurde XGBoost verwendet, da es gut für tabellarische Daten geeignet ist und nichtlineare Zusammenhänge abbilden kann. Um unterschiedliche Spielertypen besser zu berücksichtigen, wurden vier separate Modelle für Torhüter, Verteidiger, Mittelfeldspieler und Angreifer trainiert.

Auch hier wurde ein 80/20-Split verwendet. Der Split wur-

Tabelle 5: Evaluation des Next-Season-Modells nach Positionsgruppe

Position	B. MAE	XGB MAE	B. R^2	XGB R^2
Torhüter	0.1435	0.1455	-0.0024	-0.0067
Verteidiger	0.1462	0.1388	-0.0064	0.0823
Mittelfeld	0.1554	0.1412	-0.0052	0.1394
Angreifer	0.1955	0.1699	-0.0156	0.1928

de spielerbasiert durchgeführt, sodass ein Spieler entweder vollständig im Trainings- oder vollständig im Testdatensatz vorkommt. Dadurch wird verhindert, dass das Modell im Test indirekt Spieler wiedererkennt, die bereits im Training vorkamen.

Als Baseline wurde pro Positionsgruppe jeweils das durchschnittliche nächste Rating vorhergesagt. Tabelle 5 zeigt, dass XGBoost bei Verteidigern, Mittelfeldspielern und Angreifern besser abschneidet als die Baseline. Besonders bei Angreifern ist die Verbesserung am deutlichsten. Bei Torhütern funktioniert das Modell dagegen nicht besser als die Baseline. Das ist nachvollziehbar, da Torhüterleistungen mit den verfügbaren Features wie Tore, Assists oder Teamresultat nur eingeschränkt beschrieben werden.

4.4 Qualitative Prüfung des Recommenders

Neben den Metriken wurde das System auch fachlich geprüft. Dabei wurden Beispielabfragen im Recommender durchgeführt und die vorgeschlagenen Spieler mit Fussball-Domänenwissen beurteilt. Wichtig war dabei nicht nur die vorhergesagte Bewertung, sondern auch ob die Vorschläge realistisch wirken. Gerade im Amateurfussball spielen Faktoren wie Alter, Position, aktuelle Liga und Distanz zum Verein eine wichtige Rolle.

Die Vorschläge wirkten vor allem bei offensiven Spielern plausibel. Das passt zu den quantitativen Resultaten, da das Next-Season-Modell bei Angreifern die besten Werte erreichte. Als konkretes Beispiel wurde eine Abfrage nach offensiven Spielern im Alter von 18 bis 25 Jahren durchgeführt.

Gleichzeitig bleibt diese Prüfung subjektiv. Eine vollständige Evaluation des Recommenders wäre erst möglich, wenn reale Vereinsentscheidungen, Transfers oder Scouting-Feedback systematisch mit den Empfehlungen verglichen werden könnten. Für den aktuellen Prototyp zeigt die Evaluation aber, dass die Pipeline funktioniert, die Modelle nutzbare Signale erzeugen und das System als Grundlage für datengetriebenes Scouting geeignet ist.

5 Diskussion, Erkenntnisse und Limitierungen

5.1 Bewertung des Systemansatzes

Rückblickend hat vor allem der Pipeline-Ansatz gut funktioniert. Die Kombination aus Airflow, ETL-Logik, PostgreSQL-Datenbank und Modellanwendung hat sich als sinnvoll erwiesen. Besonders hilfreich war, dass die einzelnen Schritte klar getrennt sind: Scraping, Verarbeitung, Speicherung, Mo-

dellanwendung und Zugriff über die API. Dadurch konnten Fehler besser eingegrenzt werden und einzelne Komponenten unabhängig angepasst werden.

Eine wichtige Designannahme war, zuerst eine stabile Datenbasis aufzubauen und das Frontend bewusst einfach zu halten. Diese Annahme hat sich bestätigt. Der grösste Aufwand lag nicht im Frontend, sondern im Sammeln, Bereinigen und Strukturieren der Daten. Das Streamlit-Frontend erfüllt für den Prototyp seinen Zweck, weil es Abfragen und Empfehlungen sichtbar macht, ohne dass zu viel Zeit in UI-Design investiert werden musste.

Auch die relationale Datenbankstruktur war passend. Die Daten sind stark strukturiert und hängen klar über Spieler, Clubs, Saisons, Spiele und Spielerleistungen zusammen. PostgreSQL war dafür ausreichend und ist aktuell nicht der limitierende Faktor. Die Datenbank umfasst rund 116 MB und liegt damit weit unter Grössenordnungen, bei denen PostgreSQL selbst problematisch wird. PostgreSQL hat keine fixe maximale Datenbankgrösse; praktische Grenzen entstehen eher durch Speicherplatz, Indizes und Performance. Eine einzelne Tabelle kann bei Standardkonfiguration bis etwa 32 TB gross werden [5].

5.2 Herausforderung Scraping

Die grösste Schwierigkeit war eindeutig das Scraping. Es dauerte länger als erwartet, erforderte viele Webzugriffe und war teilweise schwer vorhersehbar. Zusätzlich zeigte sich gegen Ende des Projekts, dass Änderungen an externen Webseiten direkte Auswirkungen auf die Scraper haben können. Für einen Prototyp ist Scraping ein praktikabler Weg, um überhaupt an Daten zu kommen. Für ein produktives System wäre es aber keine ideale Lösung.

Eine zentrale Erkenntnis ist deshalb, dass für eine Weiterentwicklung ein offizieller Datenzugang viel sinnvoller wäre. Idealerweise würde man beim Schweizer Fussballverband anfragen, ob ein Zugriff auf ihre Datenbank oder eine offizielle Schnittstelle möglich ist. Dadurch würden viele Probleme des Scrapings wegfallen, insbesondere Laufzeit, Wartung, Datenvollständigkeit und rechtliche Unsicherheit.

Dann müsste entschieden werden, ob iAM-Scout eine eigene Kopie der Daten hält oder direkt über eine Schnittstelle auf die externe Datenbank zugreift. Bei einer direkten Schnittstelle würden sich die Limitierungen eher auf Query-Performance, API-Routen, Berechtigungen und mögliche Rate Limits verschieben. Eine eigene Kopie hätte dagegen den Vorteil, dass Abfragen unabhängig vom externen System möglich wären.

5.3 Limitierungen der Daten und Modelle

Eine weitere Limitation betrifft die Datenqualität und die verfügbaren Detailstatistiken. Die Modelle können nur Informationen verwenden, die in den Daten vorhanden sind. Im Amateurfussball fehlen aber viele Leistungsmerkmale, die im Profibereich üblich sind, zum Beispiel Passqualität, Zweikampfdaten, xG-Werte oder detaillierte Eventdaten. Dadurch bleiben die vorhergesagten Bewertungsmodell immer eine Schätzung und sollten nicht als objektive Wahrheit verstanden

werden.

Das Bewertungsmodell liefert trotzdem eine sinnvolle Orientierung, ersetzt aber kein klassisches Scouting. Es basiert auf den verfügbaren Leistungsdaten und funktioniert je nach Situation und Position unterschiedlich gut. Besonders bei Positionen, deren Leistung stark über einfache Kennzahlen sichtbar wird, sind die Resultate plausibler. Bei anderen Rollen, zum Beispiel Torhütern oder defensiven Spielern, fehlen teilweise wichtige Merkmale, um die Leistung vollständig zu erklären.

Das gleiche gilt für das Recommender-System. Die Empfehlungen sind hilfreich, weil sie mehrere Faktoren wie Leistung, Position, Alter und Distanz kombinieren können. Trotzdem hängen sie stark von den verfügbaren Daten und vom Modell ab. Die Evaluation hat gezeigt, dass das Next-Season-Modell je nach Positionsgruppe unterschiedlich gut funktioniert. Besonders bei offensiven Spielern waren die Vorschläge plausibel, während andere Positionen schwieriger zu bewerten sind.

5.4 Skalierbarkeit und Betrieb

Aus technischer Sicht ist die Speicherung aktuell nicht der Engpass. Die Datenbank ist klein genug, um deutlich erweitert zu werden. Auch eine wesentlich grössere Datenmenge wäre mit PostgreSQL grundsätzlich realistisch. Der grössere Engpass liegt im aktuellen Prototyp eher beim Datenbezug. Mehr Ligen würden beim Scraping mehr Webzugriffe, längere Laufzeiten und mehr Wartungsaufwand bedeuten.

Das Deployment auf der Schul-VM und die Containerisierung mit Docker funktionierten im Vergleich dazu gut. Hier konnte Vorwissen aus einem früheren Modul genutzt werden. Dadurch war es möglich, Airflow, Datenbank und API sauber zu betreiben und die Komponenten besser voneinander zu trennen.

5.5 Lessons Learned

Eine wichtige Erkenntnis ist, dass bei datengetriebenen Projekten die Datenbeschaffung und Datenqualität oft mehr Aufwand verursachen als das eigentliche Modell. Gerade beim Scraping ist es schwierig, Aufwand und Stabilität im Voraus korrekt einzuschätzen. Für zukünftige Projekte würden wir deshalb früher prüfen, ob offizielle Datenquellen oder Schnittstellen verfügbar sind.

Zusätzlich würden wir mehr Tests für Datenqualität und Vollständigkeit einbauen. Neben klassischen Unit Tests wären automatische Checks sinnvoll, zum Beispiel ob alle erwarteten Spiele einer Liga vorhanden sind, ob Spielerleistungen vollständig geladen wurden oder ob ungewöhnlich viele fehlende Werte auftreten. Solche Tests würden helfen, Fehler im Scraping oder in der ETL-Pipeline früher zu erkennen.

Die Arbeit im Zweierteam hat insgesamt gut funktioniert. Durch die kleine Teamgrösse war die Abstimmung einfach und Entscheidungen konnten schnell getroffen werden. Gleichzeitig half eine leichte Scrum-Struktur mit Aufgabenplanung, Reviews und klaren Zuständigkeiten, weil Scraping, Datenbank, Modell und Frontend stark voneinander abhängen. Für

ein grösseres Team müsste diese Organisation wahrscheinlich formeller umgesetzt werden.

Die wichtigste Erkenntnis aus dem Projekt ist, dass bei datengetriebenen Anwendungen die Qualität und Verfügbarkeit der Daten entscheidend ist. Die eigentliche Modellierung war erst möglich, nachdem Scraping, Datenbereinigung und Datenmodellierung stabil genug funktioniert haben. Für zukünftige Projekte würden wir deshalb noch früher in Datenvalidierung, offizielle Datenzugänge und automatisierte Qualitätschecks investieren.

6 Verwandte Arbeiten

Datengetriebene Ansätze im Fussball wurden in den letzten Jahren in verschiedenen Bereichen untersucht. Ein systematischer Überblick von Ati et al. zeigt, dass Machine Learning und Multi-Criteria-Decision-Making häufig für Spielerbewertung, Spielerauswahl und Performance Prediction eingesetzt werden [6]. Diese Arbeiten fokussieren meistens auf professionelle Ligen, da dort deutlich mehr strukturierte Leistungsdaten verfügbar sind. iAM-Scout verfolgt ein ähnliches Ziel, nämlich datenbasierte Spielerbewertung und Spielerauswahl, arbeitet aber mit deutlich begrenzteren Daten aus dem Schweizer Amateurfussball.

Ein konkreter Ansatz für Scouting wird von Lacan beschrieben. Dort wird ein stacking-basiertes Deep-Learning-Modell genutzt, um Spieler mit hohem Potenzial zu identifizieren [7]. Im Vergleich dazu ist iAM-Scout weniger auf ein möglichst komplexes Modell fokussiert. Der Schwerpunkt liegt stärker auf der gesamten Pipeline: Datenbeschaffung, Bereinigung, Speicherung, Modellanwendung und Nutzung über ein Frontend. Das Modell ist also nur ein Teil des Systems und nicht der alleinige Beitrag.

Auch die Vorhersage zukünftiger Spielerentwicklung ist ein relevantes verwandtes Thema. Van Arem et al. untersuchen, wie sich Qualität und Marktwert von Profispieler in der Zukunft entwickeln können [8]. Dies ist ähnlich zur Idee des Next-Season-Modells in iAM-Scout, bei dem eine erwartete Bewertung für die nächste Saison geschätzt wird. Der Unterschied liegt erneut in der Datengrundlage: Während solche Arbeiten meist auf professionellen Daten mit hoher Qualität basieren, muss iAM-Scout mit öffentlich verfügbaren und gröberen Amateurdaten arbeiten.

Eine weitere verwandte Arbeit ist FootballDB, bei der Fussballdaten in relationalen Datenmodellen strukturiert und für Text-to-SQL-Systeme genutzt werden [9]. Diese Arbeit zeigt, wie wichtig ein gutes Datenmodell für spätere Abfragen ist. Auch iAM-Scout nutzt eine relationale Datenbank, verfolgt aber ein anderes Ziel. Statt natürliche Sprache in SQL zu übersetzen, baut iAM-Scout zuerst eine eigene Datenbasis für den Schweizer Amateurfussball auf und verwendet diese für Ratings, Abfragen und Empfehlungen.

Zusammengefasst liegt iAM-Scout zwischen mehreren bestehenden Forschungsrichtungen: Fussballdatenbanken, Spielerbewertung, Performance Prediction und Scouting-Systeme. Der Hauptunterschied besteht darin, dass der Fokus nicht nur auf einem Modell liegt, sondern auf einer vollständigen und

reproduzierbaren Datenpipeline für einen Bereich, in dem strukturierte Daten bisher nur eingeschränkt verfügbar sind.

7 Fazit und Ausblick

Mit iAM-Scout wurde ein Prototyp entwickelt, der zeigt, wie datengetriebenes Scouting auch im Schweizer Amateurfussball umgesetzt werden kann. Das wichtigste Ergebnis ist dabei nicht nur ein einzelnes Modell, sondern das Zusammenspiel aus Scraping, ETL-Pipeline, relationaler Datenbank, Modellanwendung, API und Frontend. Dadurch konnten öffentlich verfügbare Fussballdaten gesammelt, strukturiert gespeichert und für konkrete Scouting-Fragen nutzbar gemacht werden.

Besonders wertvoll war der Aufbau der durchgängigen Datenpipeline. Die historischen Daten wurden einmalig importiert, während neue Spiele über Airflow inkrementell ergänzt werden können. Die Datenbank bildet Spieler, Clubs, Saisons, Spiele, Kader und Spielerleistungen ab und schafft damit die Grundlage für spätere Analysen. Zusätzlich konnten mit dem Ratingmodell und dem Recommender erste datenbasierte Einschätzungen und Transferhinweise erzeugt werden.

Das Projekt hat aber auch gezeigt, dass die Datenbeschaffung im Amateurfussball die grösste Herausforderung ist. Web Scraping war für den Prototyp sinnvoll, ist aber für einen langfristigen produktiven Einsatz nur bedingt geeignet. Der wichtigste nächste Schritt wäre deshalb ein offizieller Datenzugang, zum Beispiel über den Schweizer Fussballverband oder eine andere verlässliche Schnittstelle. Dadurch würden viele Probleme wie lange Scraping-Laufzeiten, Änderungen an Webseiten und Unsicherheiten bei der Datenvollständigkeit deutlich reduziert.

Mit einem stabileren Datenzugang könnte iAM-Scout deutlich erweitert werden. Naheliegend wäre die Integration weiterer Ligen, da die Architektur grundsätzlich darauf ausgelegt ist. Auch die Modelle könnten weiterentwickelt werden. Neben klassischen Machine-Learning-Verfahren wären komplexere Ansätze wie Deep Neural Networks oder zeitbasierte Modelle denkbar, da Spielerleistungen über Saisons hinweg eine zeitliche Entwicklung abbilden.

Ein weiterer wichtiger Schritt wäre die Evaluation mit echten Vereinen. Dadurch könnte geprüft werden, ob die vorgeschlagenen Spieler nicht nur aus Datensicht, sondern auch aus praktischer Sicht relevant sind. Besonders spannend wäre ein Test mit einem Amateurverein oder im Jugendbereich. Dabei müsste jedoch berücksichtigt werden, dass die Datenqualität bei tieferen Ligen oder jüngeren Spielern wahrscheinlich abnimmt.

Insgesamt war iAM-Scout ein sehr lehrreiches Projekt. Es hat gezeigt, dass datengetriebenes Scouting im Amateurbereich grundsätzlich möglich ist, wenn eine saubere Pipeline und ein sinnvolles Datenmodell vorhanden sind. Gleichzeitig wurde klar, dass gute Modelle stark von guter Datenqualität abhängen. Der entwickelte Prototyp bildet damit eine solide Grundlage, auf der ein produktiveres und breiter einsetzbares Scouting-System weiterentwickelt werden könnte.

Literatur

- [1] Unsplash. “Fussballer, die in der Nähe des Tores um den Ball kämpfen,” besucht am 26. Feb. 2026. Adresse: <https://unsplash.com/de/fotos/fussballer-die-in-der-nahe-des-tores-um-den-ball-kampfen-OH-plkdPdSU>.
- [2] Schweizerischer Fussballverband. “Qualifikationen und Transfers der Spieler/innen mittels clubcorner.ch,” besucht am 28. Apr. 2026. Adresse: <https://org.football.ch/spielerkontrolle/informationen.aspx>.
- [3] Transfermarkt. “Transfermarkt,” besucht am 28. Apr. 2026. Adresse: <https://www.transfermarkt.ch/>.
- [4] Sofascore. “Sofascore,” besucht am 28. Apr. 2026. Adresse: <https://www.sofascore.com/de>.
- [5] PostgreSQL Global Development Group. “PostgreSQL Limits,” besucht am 26. Feb. 2026. Adresse: <https://www.postgresql.org/docs/current/limits.html>.
- [6] A. Ati, P. Bouchet und R. Ben Jeddou, “Using multi-criteria decision-making and machine learning for football player selection and performance prediction: A systematic review,” *Data Science and Management*, Jg. 7, Nr. 2, S. 79–88, 2024. DOI: 10.1016/j.dsm.2023.11.001. Adresse: <https://doi.org/10.1016/j.dsm.2023.11.001>.
- [7] S. Lacan, *Stacking-based deep neural network for player scouting in football*, 2024. arXiv: 2403.08835 [cs.LG]. Adresse: <https://arxiv.org/abs/2403.08835>.
- [8] K. van Arem, F. Goes-Smit und J. Söhl, “Forecasting the Future Development in Quality and Value of Professional Football Players,” *Applied Sciences*, Jg. 15, Nr. 16, S. 8916, 2025. DOI: 10.3390/app15168916. Adresse: <https://doi.org/10.3390/app15168916>.
- [9] J. Fürst, C. Kosten, F. Nooralahzadeh, Y. Zhang und K. Stockinger, *Evaluating the Data Model Robustness of Text-to-SQL Systems Based on Real User Queries*, 2024. DOI: 10.48550/arXiv.2402.08349. arXiv: 2402.08349 [cs.DB]. Adresse: <https://arxiv.org/abs/2402.08349>.

Anhang

A Deklaration zur Nutzung von KI

Zur Erstellung dieses Textes wurden generative KI-Tools (ChatGPT 5.5) ausschliesslich zur Unterstützung bei Grammatik und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt vollständig bei den Autoren.

B Zusätzliche Modellinformationen

Für das Ratingmodell wurden numerische Variablen direkt übernommen, kategoriale Variablen per One-Hot-Encoding verarbeitet und der Random Forest über Grid Search mit 5-facher Cross-Validation optimiert. Die besten Parameter waren: `max_depth=30`, `min_samples_leaf=1`, `min_samples_split=2` und `n_estimators=1000`.